

Homework 2

Due: October 14, 2026 (11:59 PM ET)

Instructions

1. You may collaborate with one other classmate, but your final write-up and code must be your own. Please acknowledge the classmate you worked with.
2. You may refer to the lecture slides and any links posted on the course website. **DO NOT USE** any AI tools or other online resources.
3. Submit a **.zip** file containing exactly two files: **answers.pdf** and **attacks.py**. Each section below contains two types of tasks: **Implement** and **Explain**. Include your solutions to the **Implement** tasks in **attacks.py**, and your responses to the **Explain** tasks in **answers.pdf**. Submit the ZIP file through **Canvas Assignments** (NOT Gradescope). **DO NOT** include any other files in the ZIP.

Overview. As discussed in class, standard computationally secure (**or CPA-secure**) encryption schemes guarantee privacy only against **eavesdropping** adversaries. In practice, however, adversaries may be significantly more powerful: rather than merely observing the ciphertexts exchanged between Alice and Bob, they may also attempt to **tamper** with them.

CPA-secure encryption schemes do not, in general, guarantee privacy against such active adversaries. For example, suppose an adversary can modify a ciphertext sent by Alice to Bob and observe whether Bob is able to decrypt the modified ciphertext successfully. Even this seemingly limited feedback may reveal information about the underlying plaintext and, in some cases, may allow the adversary to recover the entire message. In this assignment, you will implement such attacks against **CTR-mode encryption** and **CBC-mode encryption**.

By the end of the assignment, you should understand why CPA-secure encryption schemes can be vulnerable to active adversaries who are able to tamper with ciphertexts, and why authenticated encryption motivates the stronger privacy notion of *chosen-ciphertext attack (CCA) security*.

Reference. Please refer to Chapter 9 of *The Joy of Cryptography* for additional background on these attacks: <https://joyofcryptography.com/cca/>.

Starter Code. The starter package contains **oracle.py**, **attacks.py**, **test_public.py**, and **README.md**. **You should modify only attacks.py**. The oracle implementations in **oracle.py** should be treated as black boxes. Your attack may call their documented interfaces (described below), but may not inspect private state, access secret keys, invoke instructor-only verification helpers, or otherwise bypass the oracle interface. **Your code will be tested using fresh random keys and fresh random messages.**

1 Attacks on CTR-Mode Encryption

You will implement two attacks on the following CTR-mode encryption scheme: a **bit-flipping attack** (Section 1.1) and a **format-oracle attack** (Section 1.2).

CTR-Mode Encryption. Let f_k be a *pseudorandom function (PRF)* that maps b -byte strings to b -byte strings, where the key k is also b bytes long. The counter (CTR) mode encryption scheme (as discussed in class) for encrypting variable-length messages consisting of t blocks, each of length b bytes, works as follows:

- **KeyGen** $\rightarrow k$: Sample a uniformly random b -byte key k .
- **Enc**(k, m) $\rightarrow c$: Parse $m = m_1 \parallel \dots \parallel m_t$, where $m_1, \dots, m_{t-1}$ are b -byte strings and m_t contains between 1 and b bytes. For each i , compute the b -byte keystream block $s_i = f_k(\text{ctr} \oplus i)$, and let $c_i = m_i \oplus s_i[0 : |m_i|]$. Output $c = (\text{ctr}, c_1, \dots, c_t)$.
- **Dec**(k, c) $\rightarrow m$: Parse $c = (\text{ctr}, c_1, \dots, c_t)$. For each $i \in [t]$, let i denote its b -byte representation and compute $m_i = f_k(\text{ctr} \oplus i) \oplus c_i$. Output $m = m_1 \parallel \dots \parallel m_t$.

1.1 Bit-Flipping Attack

Observe that replacing any c_i with $c'_i = c_i \oplus \Delta$ causes the corresponding message block to become $m'_i = m_i \oplus \Delta$. As a warm-up, you will first implement this bit-flipping attack.

Starter Code. The file `oracle.py` contains a `CTRTransactionService` that encrypts exactly the message `from=alice&to=bob&amount=1000` using CTR-mode encryption. You (the attacker) are given `ciphertext = service.get_transaction()` (see `test_public.py`), and you know the complete message above. You do **not** know the key.

Tasks. You need to do the following:

- **[Task 1 (10 points)]: Implement** the following in `attacks.py`:

```
def change_amount(ciphertext: bytes) -> bytes:
    ...
```

so that the returned ciphertext decrypts to `from=alice&to=bob&amount=9000`. You may not use a decryption oracle for this part.

- **[Task 2 (5 points)]: Explain** why your modification succeeds even though you do not know the encryption key.

1.2 Format-Oracle Attack

You will now implement a format-oracle attack to first recover **one byte of the message** and then the **entire message**.

Starter Code. You (the attacker) now receive a CTR ciphertext encrypting an **unknown** message. The plaintext is guaranteed not to contain the byte 0x00. You have access to the following oracle (see `oracle.py`):

```
def has_null(ciphertext: bytes) -> bool:
    """
    Returns True iff the decrypted plaintext contains byte 0x00.
    """
```

The oracle reveals only one bit of information per query.

Tasks. You need to do the following:

- **[Task 3 (15 points)]: Implement** the following in `attacks.py`:

```
def recover_ctr_byte(ciphertext: bytes, position: int, oracle) -> int:
    ...
```

The parameter `position` is a zero-based index into the plaintext. Your function should return the recovered message byte as an integer in $\{0, \dots, 255\}$. Your attack should use at most 255 oracle queries.

- **[Task 4 (5 points)]: Explain** Suppose the plaintext byte at position i is m_i . Explain what happens when you XOR the corresponding ciphertext byte with a value x . For what value of x does that plaintext byte become 0x00?
- **[Task 5 (10 points)]: Implement** the following in `attacks.py`:

```
def recover_ctr(ciphertext: bytes, oracle) -> bytes:
    ...
```

Your attack must work for arbitrary null-free binary messages; you should not assume that the message is ASCII or printable. If the message contains n bytes, your implementation should make at most $255n$ oracle queries.

- **[Task 6 (5 points)]: Explain** A naive exhaustive search over an n -byte message requires exponentially many guesses. Explain why the format oracle allows you to recover the message one byte at a time.

2 Attacks on CBC-Mode Encryption

You will now implement a **padding-oracle attack** against CBC-mode encryption. You will first **explore the malleability** of CBC-mode encryption (Section 2.1), then **recover a single message block** (Section 2.2), and finally **recover the entire message** (Section 2.3).

CBC-Mode Encryption. Let f_k be a pseudorandom permutation (PRP) that maps b -byte strings to b -byte strings, and let f_k^{-1} denote its inverse. The CBC-mode encryption scheme for encrypting messages whose length is a multiple of b bytes works as follows:

- **KeyGen** $\rightarrow k$: Sample a uniformly random key k .
- **Enc**(k, m) $\rightarrow c$: Parse $m = m_1 \parallel \dots \parallel m_t$, where each m_i is a b -byte string. Sample a uniformly random b -byte initialization vector IV . Let $c_0 = IV$. For each $i \in [t]$, compute $c_i = f_k(m_i \oplus c_{i-1})$. Output $c = (IV, c_1, \dots, c_t)$.
- **Dec**(k, c) $\rightarrow m$: Parse $c = (IV, c_1, \dots, c_t)$ and let $c_0 = IV$. For each $i \in [t]$, compute $m_i = f_k^{-1}(c_i) \oplus c_{i-1}$. Output $m = m_1 \parallel \dots \parallel m_t$.

PKCS#7 Padding. Since CBC mode operates on complete b -byte blocks, messages whose length is not a multiple of b bytes must first be padded. In this assignment, we use **PKCS#7 padding**. Suppose the message requires d additional bytes to make its length a multiple of b , where $1 \leq d \leq b$. PKCS#7 appends exactly d bytes, each having the byte value d . For example, if $d = 1$, the message ends in byte `0x01`; if $d = 2$, it ends in bytes `0x02 0x02`; and if $d = 4$, it ends in bytes `0x04 0x04 0x04 0x04`. If the original message length is already a multiple of b , an entire block of padding is added, consisting of b copies of the byte whose value is b . The starter code given to you uses AES as a PRP, where $b = 16$, this means appending 16 copies of the byte `0x10`.

Starter Code. The file `oracle.py` provides access to the following padding oracle:

```
def valid_padding(ciphertext: bytes) -> bool:
    """
    Returns True iff the resulting CBC plaintext has valid PKCS#7 padding.
    """
```

The oracle returns only a single bit indicating whether the decrypted message has valid PKCS#7 padding. It never reveals the message itself.

2.1 Understanding CBC Malleability

Suppose two consecutive ciphertext blocks are c_{i-1} and c_i , then we know that $m_i = f_k^{-1}(c_i) \oplus c_{i-1}$. Now, let's replace c_{i-1} with $c'_{i-1} = c_{i-1} \oplus \Delta$.

Tasks. You need to do the following:

- **[Task 7 (5 points)]: Explain** why the resulting message block is $m'_i = m_i \oplus \Delta$. Why does this allow an attacker to manipulate the decryption of c_i without knowing either k or $f_k^{-1}(c_i)$?
- **[Task 8 (5 points)]: Explain** how two consecutive ciphertext blocks $c_{i-1} \parallel c_i$ can be treated as a standalone CBC ciphertext for recovering the message block corresponding to c_i .

2.2 Padding Oracle Attack (Recovering One Message Block)

You will now use the padding oracle to recover one complete message block.

Tasks. You need to do the following:

- [Task 9 (20 points)]: **Implement** the following in `attacks.py`:

```
def recover_cbc_block(previous_block: bytes, block: bytes, oracle) -> bytes:
    ...
```

Both `previous_block` and `block` are exactly 16 bytes long. Your function should return the plaintext block $f_k^{-1}(\text{block}) \oplus \text{previous_block}$.

As a useful first step, consider how to recover only the final byte of the plaintext block. In particular, think about how you can modify the final byte of `previous_block` so that the final byte of the resulting plaintext becomes `0x01`. Once you have recovered the final plaintext byte, determine how to modify the previous ciphertext block so that the final two plaintext bytes become `0x02 0x02`, and continue similarly to recover the remaining bytes.

Important edge case: Your implementation must correctly handle cases in which more than one candidate modification produces valid padding. The hidden test cases include messages whose suffixes may accidentally resemble valid PKCS#7 padding. Therefore, you should **not** assume that the first successful guess is necessarily correct. A straightforward implementation should require at most approximately $256 \cdot 16$ oracle queries per block, plus a small number of additional queries to resolve ambiguous responses.

2.3 Padding Oracle Attack (Recovering the Entire Message)

You will now extend your attack to recover an entire encrypted message.

Tasks. You need to do the following:

- [Task 10 (20 points)]: **Implement** the following in `attacks.py`:

```
def recover_cbc(ciphertext: bytes, oracle) -> bytes:
    ...
```

The input ciphertext has the form $IV \| c_1 \| c_2 \| \dots \| c_n$. Your function should recover every message block, then concatenate the recovered blocks, remove the PKCS#7 padding, and finally return the original unpadded message.

Your attack must work for arbitrary binary messages; you should not assume that the message is ASCII or printable. The hidden test cases will include: one-block messages, multi-block messages, every PKCS#7 padding length from 1 through 16, arbitrary binary messages, messages whose suffixes resemble valid PKCS#7 padding, and messages requiring a full 16-byte padding block.